Structural Privacy Vulnerabilities in Quantum Neural Networks

Anonymous Author(s)
Affiliation
Address
email

Abstract

Quantum neural networks (QNNs) are increasingly considered for near-term learning tasks involving sensitive data, yet the influence of architectural design choices on their privacy risk remains poorly understood. Prior work has shown that QNNs can leak membership information, but it remains unclear where this leakage originates within the QNN pipeline. Is privacy risk primarily a consequence of the trainable ansatz, or can it be shaped upstream by the non-trainable encoder and its induced Hilbert-space geometry? We study this question through QuRiFT (Quantum Risk and Inference Fault-line Tracer), a controlled audit framework for structural privacy analysis in QML. Our central hypothesis is that the classical-to-quantum encoder shapes the Hilbert-space geometry on which the variational ansatz operates, inducing train–test asymmetries that later become exploitable by membership inference attacks. Using QuRiFT, we perform controlled interventions across QNN, HQNN, and QCNN models by varying feature-map family, data-reuploading-style feature-map repetitions, circuit width, variational depth, and variational-layer design while keeping training protocols fixed. The results reveal a clear structural hierarchy in our controlled regimes: privacy risk is driven more strongly by the data-encoding circuit than by trainable variational depth. In particular, `Z` and `ZZ` feature maps induce larger generalization gaps and stronger membership signals than `effSU2`. Feature-map repetitions and circuit width act as leakage amplifiers, with repetitions correlating strongly with larger train–test gaps ($r = 0.74$) and lower test accuracy ($r = -0.83$), producing a privacy–utility anti-tradeoff. Direct membership inference validates the structural signal: high-risk configurations identified by QuRiFT reach attack AUCs up to 0.870, compared with 0.530 for low-risk baselines. These findings show that the encoder should be treated as a first-order privacy parameter in QML design, with feature-map choice and data re-uploading audited as security-relevant architectural decisions rather than routine expressivity knobs.

1 Introduction

Modern learning systems are increasingly trained on data whose mere inclusion in a training set can itself be sensitive. A medical record can imply a diagnosis or treatment history; a financial record can point to credit risk or fraud exposure; a government record can disclose employment, clearance, or background-screening information. Recent incidents make this risk concrete rather than hypothetical: the Change Healthcare incident involved protected health information, the Capital One breach exposed information about credit-card customers and applicants, and the U.S. Office of Personnel Management breach compromised 21.5 million personnel and background-investigation records at national scale [9, 28, 29]. However, for machine learning, the concern is not limited to direct database compromise. Even when raw records are never released, trained models can leak information

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

about their training data through outputs, losses, gradients, or confidence patterns [5, 25, 27, 33]. Membership inference attacks (MIAs) ask whether an adversary can tell if a particular record was used to train a model. In settings such as healthcare, finance, genomics, and public-sector analytics, the answer to that question can itself be a critical privacy violation.

Quantum machine learning (QML) is increasingly explored for data-intensive and sensitive domains [3, 8, 32]. Near-term QML models are typically built from parameterized quantum circuits (PQCs), which form the backbone of many quantum neural networks (QNNs) and variational quantum classifiers (VQCs) [2, 3]. This pipeline differs from classical neural networks in a fundamental way: before optimization begins, a classical input x is embedded into a quantum state through a fixed data-encoding circuit, or feature map [11, 18, 24]. This encoder $U_\phi(x)$ maps the input to a quantum feature state $\rho_\phi(x) = U_\phi(x)\rho_0 U_\phi(x)^\dagger$, inducing a Hilbert–Schmidt kernel $K_\phi(x_i, x_j) = \mathrm{Tr}(\rho_\phi(x_i)\rho_\phi(x_j))$ that determines the geometry of the quantum feature space before the variational ansatz is optimized. The model therefore operates on a representation shaped by architectural choices made prior to training, rather than one evolving solely through iterative optimization.

This suggests a different way to characterize privacy leakage in QML. In classical deep learning, memorization is typically viewed as an emergent property of over-parameterization, optimization dynamics, and the model's capacity to fit rare or noisy training samples [1, 4, 10, 34]. In a QNN, however, a non-trivial portion of the privacy-relevant structure is frozen prior to the first training iteration. Because the encoder pre-determines the proximity and local sensitivity of classical samples within the induced Hilbert space, it may inherently embed vulnerabilities before any optimization occurs. This raises a more fundamental question than whether QNNs can leak membership information: **To what extent does the non-trainable encoder itself induce the train-test asymmetry that membership inference attacks (MIAs) later exploit?** While recent studies have examined QML privacy through the lenses of differentially private training [31], unlearning [26], and gradient-based reconstruction [12], a structural account of leakage remains missing. Specifically, it remains unclear how architectural primitives, including feature-map family, encoder repetitions, circuit width, variational depth, and entanglement topology, shape the empirical privacy risk of the trained model.

We study this problem as *structural privacy vulnerability*, where privacy leakage is shaped not only by training dynamics or attack strength, but also by architectural choices made before learning begins. Our central hypothesis is that the data encoder shapes the geometry of the quantum feature space in ways that can make certain configurations more prone to overfitting, which in turn strengthens membership signals in the model outputs. To study this effect, we introduce QuRiFT (Quantum Risk and Inference Fault-line Tracer), a diagnostic framework for auditing how specific QML design choices affect membership leakage. Rather than treating privacy risk only as a post-training issue to be mitigated or removed [26], QuRiFT examines which architectural components lead to the overfitting patterns from which leakage emerges. We evaluate this question across Quantum Neural Networks (QNNs), Hybrid QNNs (HQNNs), and Quantum Convolutional Neural Networks (QCNNs), using both synthetic and benchmark datasets. By varying encoder type, circuit width, feature-map repetitions, variational depth, entanglement structure, and gate choices under a unified training protocol, we identify the structural factors most closely associated with memorization and membership leakage.

Our results show a clear hierarchy of risk. In many settings, the strongest driver of leakage is the data-encoding strategy and its repetition, rather than the depth of the trainable variational circuit. The `Z` and `ZZ` feature maps produce substantially larger train–test gaps than more stable alternatives such as `effSU2`. A variance-attribution analysis further shows that the feature-map family explains a sizeable share of the generalization-gap variance, accounting for 22.5% in the general setting and up to 44% in data-scarce regimes. We also find that utility and privacy can degrade together. Increasing the number of feature-map repetitions is strongly associated with larger generalization gaps ($r = 0.74$) and lower test accuracy ($r = -0.83$), suggesting that additional encoding complexity can make the model both less accurate and more exposed to membership inference. Direct membership inference attacks follow the same pattern, with high-risk configurations yielding attack AUCs as high as 0.870, compared with 0.530 for low-risk baselines.

Our **contributions** are fourfold. **(1)** We formalize membership leakage as a circuit-design problem rather than a byproduct of optimization alone. We demonstrate that specific architectural choices can predispose a QNN to memorize data, independent of the optimizer or training stochasticity. **(2)** We identify the classical-to-quantum feature map as a major driver of privacy risk, showing that the

encoder-induced Hilbert–Schmidt geometry can create train–test separation that MIAs exploit. **(3)** We introduce QuRiFT, a diagnostic framework for auditing utility and leakage across feature-map families, repetitions, circuit width, variational depth, and entanglement topology. **(4)** We validate the resulting structural risk patterns using MIAs, showing that high-risk configurations translate into exploitable leakage. We further find that repeated data encoding can reduce test accuracy while increasing membership vulnerability, indicating that utility and privacy can degrade together.

2 Preliminaries

A variational quantum classifier consists of a fixed data encoder followed by a trainable variational circuit. Given an input x, the encoder $U_\phi(x)$ maps an initial state ρ_0 to

$$\rho_\phi(x) = U_\phi(x)\rho_0 U_\phi(x)^\dagger. \tag{1}$$

The trainable circuit U_θ then acts on this encoded state, and class probabilities are obtained by measurement. Thus, before any optimization occurs, the model already commits to a quantum representation of the data.

The encoder induces a similarity structure through the Hilbert–Schmidt kernel

$$K_\phi(x_i, x_j) = \mathrm{Tr}(\rho_\phi(x_i)\rho_\phi(x_j))\,. \tag{2}$$

Different choices of U_ϕ therefore change how the same classical dataset is arranged in quantum feature space, affecting which samples are close, separated, or locally sensitive before the ansatz is trained. Prior work has shown that encoding choices shape the hypothesis class, decision boundary, robustness, and generalization of parameterized quantum circuits [6, 11, 20, 24]. We study the privacy consequence of this observation: if an encoder induces a representation that makes training samples easier to fit than unseen samples, it can create the train–test asymmetry later exploited by membership inference attacks. We therefore treat the feature-map family (`fm_kind`) and data-reuploading-style repetitions (`reps`) as privacy-relevant structural variables, rather than as routine encoding hyperparameters.

We measure this asymmetry using the train–test accuracy gap $\Delta_{\mathrm{gen}} = \mathrm{Acc}_{\mathrm{train}} - \mathrm{Acc}_{\mathrm{test}}$, and validate it through output-based MIAs [5, 25, 33]. In all experiments, inputs are normalized and loaded through rotation-based angle encoding; implementation details, including tiled feature-map encoding and padding, are given in Appendix A.3.

3 Hypothesis and Controlled Experimental Design

Our central hypothesis is that membership leakage in variational quantum classifiers is shaped upstream by the classical-to-quantum encoder. The encoder does not merely load data into a circuit; it induces a Hilbert–Schmidt kernel geometry over the data distribution, determining which samples become close, separated, or locally sensitive before the variational ansatz is optimized. We therefore study the privacy leakage pathway as

$$U_\phi(x) \to \rho_\phi(x) \to K_\phi \to \text{Hilbert–Schmidt kernel geometry} \to \Delta_{\mathrm{gen}} \to \mathcal{L}_{\mathrm{MIA}}.$$

The claim is not that membership leakage bypasses overfitting. Rather, we argue that in QML, overfitting can be conditioned by a fixed, non-trainable encoding layer. This is the point of departure from the usual classical view, where memorization is primarily associated with trainable parameters, model size, and optimization dynamics.

This framing yields three testable predictions. First, changing U_ϕ while holding the training protocol and model family fixed should systematically change the train–test gap. Second, increasing feature-map repetitions should amplify the gap more strongly than increasing variational depth, because repetitions repeatedly inject data-dependent structure into the quantum feature state. Third, configurations with larger gaps should generally produce stronger membership inference signals, although the gap need not be a deterministic predictor of attack success.

We test these predictions using QuRiFT [7], a controlled profiling pipeline for structural privacy analysis in QML; the QNN training and privacy-analysis flow is shown in Appendix B. QuRiFT is built on TorchQuantum [30], whose PyTorch-native circuit primitives support differentiable simulation

and GPU-accelerated training through the PyTorch execution stack. For the encoding layer, QuRiFT uses TorchQuantum-compatible implementations of standard Qiskit-style feature-map templates, including Z, ZZ, and Pauli feature maps [14–16, 23]. The full modular software organization is described in Appendix C. The goal is not to run a broad hyperparameter benchmark. Instead, QuRiFT performs controlled interventions on the QML pipeline: it varies the encoder family, feature-map repetitions, circuit width, variational depth, and variational-layer design while keeping the training protocol fixed within each experimental family. All reported experiments use controlled noiseless TorchQuantum simulation on DGX/A100 servers, which is appropriate for isolating the privacy effect of the fixed encoder U_ϕ before introducing backend-specific hardware noise as a separate deployment variable. This design lets us test whether privacy risk tracks specific circuit components, rather than treating leakage as an incidental consequence of training stochasticity or uncontrolled implementation differences.

For each model family, we sweep the main structural components of the QML pipeline:

$$\texttt{fm\_kind}, \quad \texttt{reps}, \quad \texttt{n\_wires}, \quad \texttt{depth}, \quad \texttt{ql\_ent}, \quad \texttt{ql\_op}.$$

Here, `fm_kind` denotes the feature-map family, including Z, ZZ, and `effSU2`; `reps` is the number of feature-map repetitions; `n_wires` is the number of qubits; `depth` is the variational depth; and `ql_ent` and `ql_op` specify the entanglement topology and two-qubit gate family in the trainable circuit. The key comparison is between data-reuploading-style feature-map repetitions and increased variational depth, since these two forms of scaling are often conflated but affect different parts of the pipeline: repetitions repeatedly inject input-dependent structure through the fixed encoder, whereas depth increases the trainable ansatz capacity.

We evaluate synthetic binary benchmarks, including *Moons*, *Circles*, and *Blobs*, together with a four-class MNIST subset over digits $\{0, 1, 3, 8\}$. For MNIST, each model family receives a compact 1×16 representation before the main quantum encoder, but this representation is produced differently across architectures: the dense QNN uses non-trainable PCA, bilinear pooling, or average pooling; the HQNN uses a trainable CNN preprocessor motivated by prior work [26]; and the QCNN uses a quanvolutional preprocessor [30]. In HQNN and QCNN, measured quantum features are further post-processed by an MLP head before final classification. These architectures allow us to test whether architectural inductive bias can dampen encoder-induced overfitting.

For every configuration, QuRiFT records train, validation, and held-out test accuracy and loss. It also exports prediction vectors and derived statistics, including loss, entropy, confidence, margin, and correctness, for downstream membership inference analysis. We first use Δ_{gen} to identify structurally high-risk configurations, then run MIAs on selected baseline, stress, and hard regimes to test whether the structural gap translates into measurable leakage. Full implementation details, including the simulation backend and architecture-specific preprocessing, are given in Appendix A.

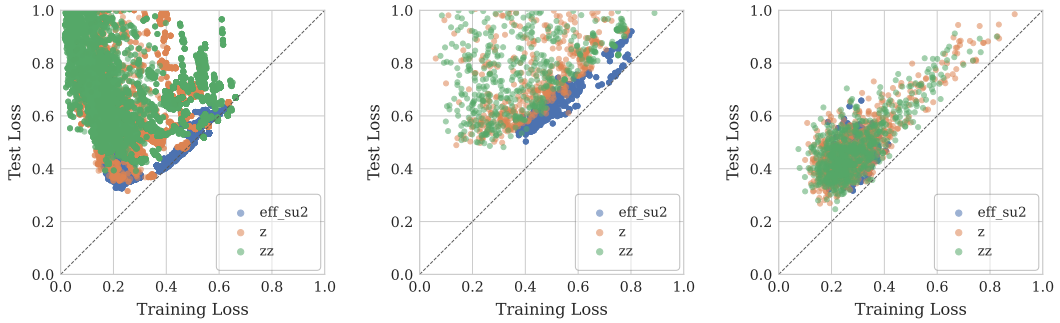
For MIA validation, we select three regimes from the controlled sweeps: a *baseline* reference configuration, a *stress* configuration with stronger structural overfitting, and a *hard* configuration with stronger held-out utility and a smaller or more controlled gap. These regimes are not new objectives; they are post-hoc selections used to test whether structural overfitting translates into measurable leakage.

4 Encoder-Induced Overfitting

We first test whether the encoder alone can stratify generalization behavior. Fig. 1 plots training loss against test loss across representative synthetic and MNIST settings, grouped by feature-map family. Points near the diagonal indicate similar train and test behavior, while points displaced above the diagonal indicate overfitting.

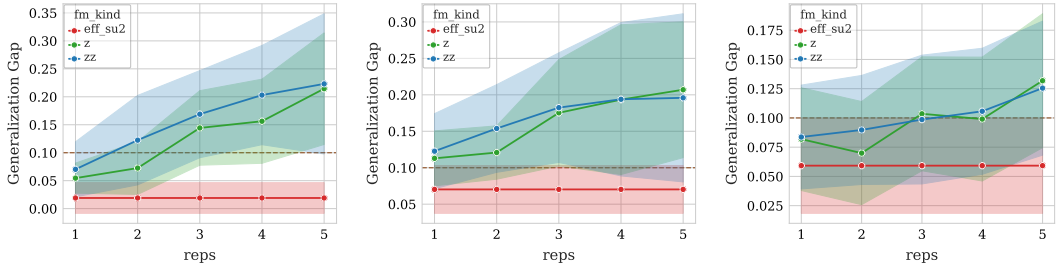
A clear pattern emerges. Configurations using Z and ZZ are more frequently displaced away from the train–test diagonal, indicating larger generalization gaps. In contrast, `effSU2` configurations remain closer to the diagonal, suggesting more stable generalization. This supports the first prediction: the encoder is not merely a preprocessing choice, but a structural component that changes the Hilbert-space geometry on which the variational ansatz acts.

We next examine whether this effect grows when the encoder is repeated. Fig. 2 shows that increasing feature-map repetitions amplifies the generalization gap for Z and ZZ, while `effSU2` remains comparatively stable. This matters because repetitions do not simply add trainable parameters. They increase



(a) Moon–QNN (b) MNIST–QNN (c) MNIST–QCNN

Figure 1: Feature-map choices induce distinct overfitting regimes across QML models. The Moon dataset provides a controlled nonlinear example, while the MNIST panels show that encoder-dependent behavior persists across QNN and QCNN architectures. Additional synthetic and HQNN panels are reported in Appendix E.



(a) Moons (b) MNIST–QNN (c) MNIST–QCNN

Figure 2: Generalization gap as a function of feature-map repetitions (`reps`). Increasing `reps` amplifies the gap for `Z` and `ZZ`, whereas `effSU2` remains comparatively stable. Additional synthetic results and the MNIST–HQNN panel are reported in Appendix F.

the complexity of the data-dependent part of the circuit. The privacy-relevant capacity is therefore introduced before the trainable ansatz, through repeated re-uploading of classical information into the quantum feature state.

Together, these results establish the first link in the structural privacy pathway:

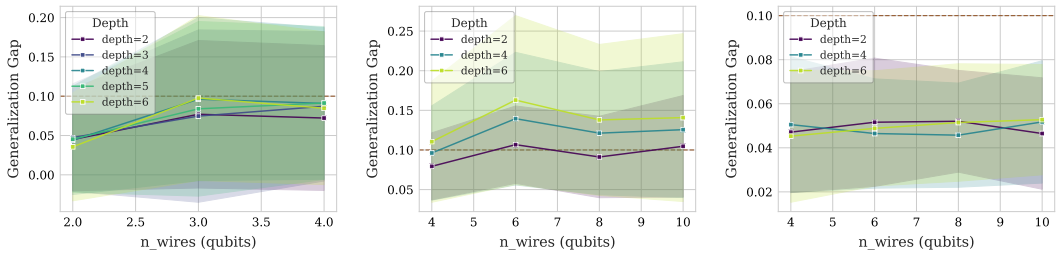
$$U_\phi(x) \to \rho_\phi(x) \to K_\phi \to \Delta_{\text{gen}}.$$

The important point is that the train–test discrepancy is not introduced only by the trainable ansatz. It can emerge from the geometry induced by the non-trainable encoder itself. At this stage, we have not yet claimed that membership leakage has occurred; rather, we have identified the structural condition that MIAs exploit. Section 8 closes this loop by testing whether these high-gap regimes produce stronger attack signals.

5 Scaling and Interaction Effects

We next examine whether the observed behavior is explained by conventional scaling factors. Fig. 3 shows that increasing circuit width can increase the generalization gap, particularly in deeper circuits. This is expected because larger Hilbert spaces allow more expressive decision boundaries. However, width does not replace the encoder effect; rather, it often amplifies the behavior already introduced by the feature map and its repetitions.

A more revealing comparison emerges when separating feature-map repetitions from variational depth. Fig. 4(left) shows that repetitions are more strongly associated with the generalization gap ($r = 0.74$) than depth ($r = 0.41$), and the same ordering holds for attack AUC and fixed-FPR leakage. At the same time, repetitions strongly reduce test accuracy ($r = -0.83$), while depth has almost no corresponding trend ($r = -0.04$).



(a) Moons (b) MNIST–QNN (c) MNIST–HQNN

Figure 3: Generalization gap as a function of circuit width (`n_wires`) across depths. Widening generally increases the gap, and the increase is typically stronger in deeper circuits. The effect is visible in both a controlled synthetic setting and MNIST-based architectures, while HQNN shows a more moderated dependence on width. Additional synthetic results and the MNIST–QCNN panel are reported in Appendix G.

Attack/utility metric Y	$r(\texttt{reps}, Y)$	$r(\texttt{depth}, Y)$
Generalization gap Δ_{gen}	0.74	0.41
Attack AUC	0.53	0.40
TPR @0.5 FPR	0.58	0.41
TPR @0.1 FPR	0.56	0.42
Target test accuracy	-0.83	-0.04
Pearson $r(\texttt{reps}, \texttt{depth})$	0.00	

Key observation. Feature-map repetitions are more strongly associated with overfitting and membership leakage than variational depth, while also correlating strongly and negatively with test accuracy. This indicates a privacy–utility anti-tradeoff: repeated data encoding can reduce utility while increasing leakage.

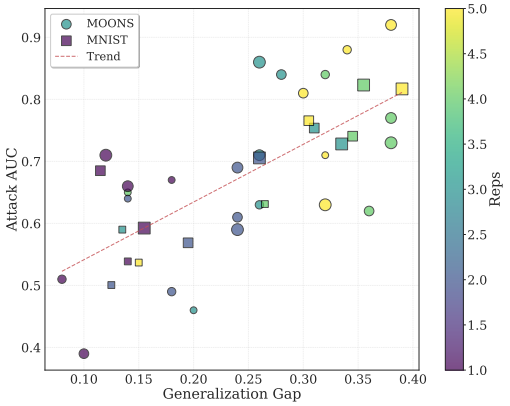


Figure 4: Correlation-based validation of structural privacy risk. Left: Pearson correlation (r) between structural scaling variables and utility/privacy metrics across $N = 40$ MNIST–QNN configurations; entries are correlation coefficients, not raw metric values. Right: larger train–test gaps are associated with stronger membership inference, supporting Δ_{gen} as a structural privacy-risk proxy.

This exposes a *privacy–utility anti-tradeoff*. Increasing feature-map repetitions does not simply buy additional expressivity at a privacy cost; in our sweep it often reduces test accuracy while strengthening the membership signal. This behavior is sharper for repeated data encoding than for variational depth, indicating that the risky scaling occurs in the data-dependent part of the circuit rather than only in the trainable ansatz. Repetitions therefore behave as a leakage amplifier, not merely as a capacity knob. Additional utility trends for width, repetitions, and depth are reported in Appendix H.

6 Negative Controls: Variational-Layer Design

To test whether all architectural choices produce comparable effects, we vary lower-level variational-layer design choices: entanglement topology and two-qubit gate family. Fig. 5 shows that these choices introduce only modest variation in the generalization gap compared with the stronger effects of feature-map design, repetitions, and circuit width.

This negative control is central to the claim. If every circuit knob produced a comparable shift in the generalization gap, the result would be only a broad hyperparameter sensitivity study. Instead, the weak dependence on `ql_ent` and `ql_op` shows that the dominant privacy signal is not arbitrary circuit variation. It is concentrated in the representation-forming part of the model: the encoder, its repetitions, and the scaling factors that amplify their effect. Detailed FM–QL interaction values are reported in Appendix I.

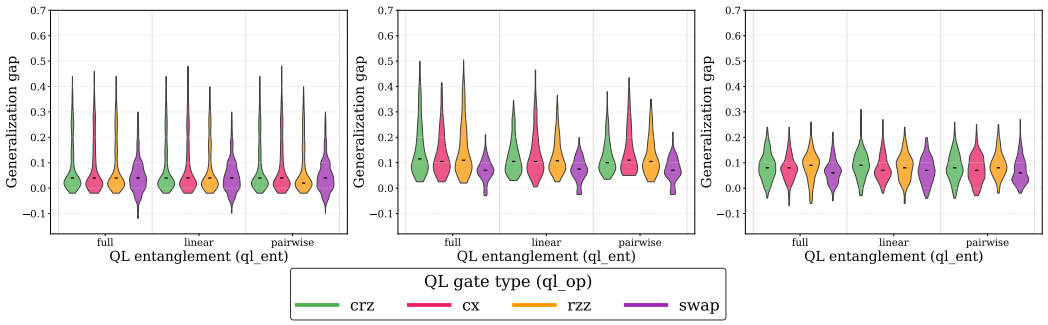


Figure 5: Interaction between variational-layer entanglement topology and two-qubit gate type on Moon–QNN, MNIST–QNN, and MNIST–QCNN from left to right, respectively. The generalization-gap distributions remain broadly similar across `ql_ent` choices (`full`, `linear`, `pairwise`) and `ql_op` choices (`crz`, `cx`, `rzz`, `swap`). These lower-level circuit choices introduce only modest variation compared with the stronger effects of feature-map design, repetitions, and circuit width. Additional results are reported in Appendix I.

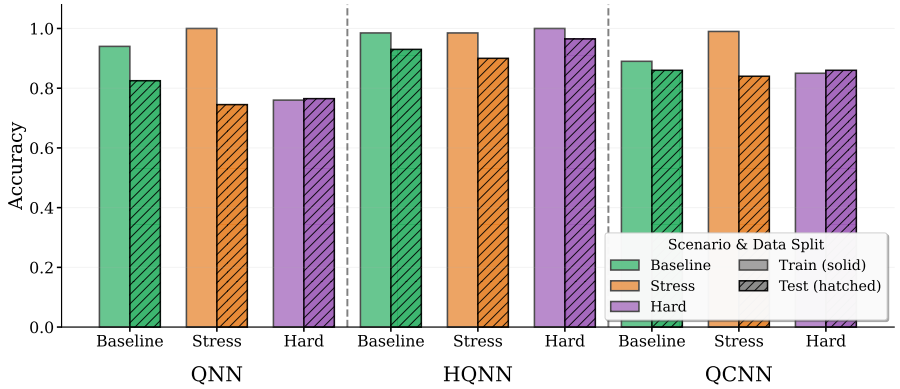


Figure 6: MNIST: matched baseline–stress–hard configurations across QNN, QCNN, and HQNN architectures. Within each regime, the circuit setting is fixed across architectures, while the architecture wrapper and its front-end/head differ. Dense QNN uses non-trainable compression, HQNN uses a CNN bottleneck followed by an MLP head, and QCNN uses a quantum-filter front-end followed by an MLP head.

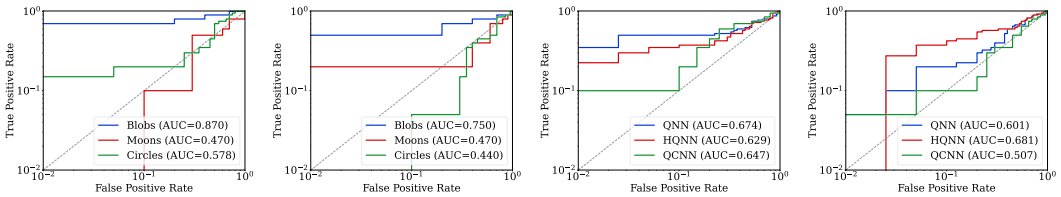
7 Architecture-Level Dampening Effects

We next test whether architecture dampens encoder-driven overfitting. Fig. 6 compares QNN, HQNN, and QCNN under matched baseline, stress, and hard regimes: within each regime, (`fm_kind`, `reps`, `n_wires`, `depth`, `ql_ent`, `ql_op`) is fixed, and only the architecture wrapper changes. QNN shows the largest stress-regime gap, while HQNN and QCNN are more stable.

These results show that structural privacy risk is modulated by architectural inductive bias. The HQNN bottleneck reduces the direct exposure of raw inputs to the quantum encoder, while QCNN-style locality constrains how information is distributed across the circuit. Thus, the encoder is not a universal one-factor explanation. Its privacy impact is mediated by the architecture in which it is embedded. This strengthens the structural account that leakage is not an arbitrary artifact of training, but a property of how encoding, circuit scaling, and architectural bias interact. Results for Synthetic baseline–stress–hard configurations are reported in Appendix J.

8 Empirical Validation via Membership Inference Attacks

We now test whether the high-gap regimes identified by QuRiFT translate into membership leakage. We use a black-box MIA setting in which the adversary queries the trained target QML model as an



(a) Synthetic–stress (b) Synthetic–hard (c) MNIST–stress (d) MNIST–hard

Figure 7: Log-scale ROC curves for MIA under structurally selected stress and hard regimes. Stress configurations generally produce stronger attack curves, while hard configurations reduce attack advantage, confirming that the structural factors identified earlier translate into measurable membership leakage.

oracle and observes only the full prediction vector for an input sample; the attacker has no access to parameters, gradients, circuit states, training dynamics, or internal measurements. Following output-based MIAs [5, 25, 33] and the QML membership-leakage setting used in quantum machine-unlearning work [26], we derive attack features from the prediction vector, including confidence, entropy, margin, loss, and correctness when the candidate label is known. Fig. 7 reports log-scale ROC curves for structurally selected stress and hard configurations.

The clearest case appears on Blobs–QNN, where the stress configuration reaches an attack AUC of 0.870, compared with 0.530 for the baseline. Across the broader set, stress configurations often increase attack strength, although exceptions such as Moons–QNN and MNIST–HQNN show that the generalization gap is a strong proxy rather than a deterministic predictor. Full attack metrics are reported in Appendix L.

The factor-level MI attack trends are consistent with the same interpretation. Fig. 4(left) shows that feature-map repetitions are more strongly associated with attack AUC and fixed-FPR true-positive rates than variational depth. Fig. 4(right) further shows a positive association between the train–test gap and attack AUC. The full correlation matrix in Appendix N reports a Pearson correlation of $r = 0.70$ between `target_gap_acc` and `attack_auc`.

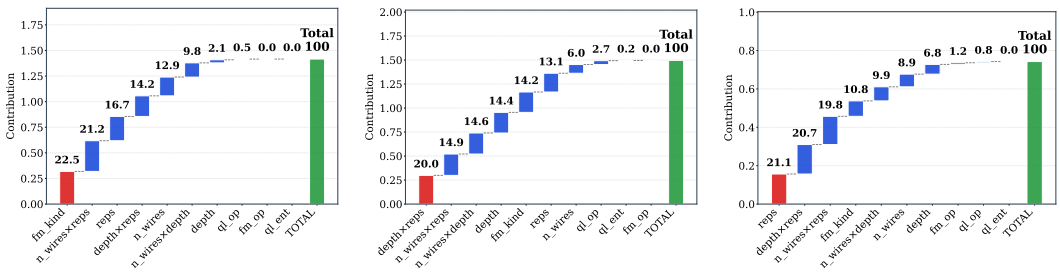
These results support the use of the generalization gap as a structural privacy-risk proxy, while leaving room for exceptions. Attack success is not determined by the gap alone; it also depends on the measurement-induced posterior geometry, class-confidence structure, and separability of member and non-member output distributions. Nevertheless, the positive correlation confirms the main empirical thesis: architectural choices that amplify memorization are generally associated with stronger measurable privacy leakage.

Taken together, these results clarify the role of the encoder. The encoder does not reveal membership by itself; it shapes the quantum representation on which training operates. When that representation induces a larger train–test asymmetry, the asymmetry becomes visible in the measurement-induced posterior behavior and can be exploited by MIAs. This is the empirical link between encoder design and privacy leakage.

9 Attribution and Privacy-Aware Design Implications

We quantify the contribution of architectural factors to the generalization gap using factor-attribution analysis. Fig. 8 shows that feature-map design, feature-map repetitions, circuit width, and their interactions explain most of the variation in Δ_{gen}, while variational-layer choices contribute much less.

The attribution results turn the qualitative trends into a component-level privacy map. Across the representative settings, the dominant sources of variation in Δ_{gen} are the feature-map family, feature-map repetitions, circuit width, and their interactions. In contrast, variational-layer entanglement and two-qubit gate choices contribute little. This separation is important because the privacy risk is concentrated in the representation-forming and scaling components of the QNN pipeline, not uniformly distributed across all circuit choices.



(a) Moons (b) MNIST–QNN (c) MNIST–QCNN

Figure 8: Factor-wise attribution of structural components to the generalization gap. Feature-map design, feature-map repetitions, circuit width, and their interactions explain most of the variance in the gap, while variational-layer choices such as entanglement topology and two-qubit gate type contribute little. Additional attribution panels for Blobs, Circles, and MNIST–HQNN are reported in Appendix O.

The HQNN case adds nuance rather than weakening the claim. Its classical bottleneck reduces the direct contribution of feature-map family and shifts variation toward width-related factors. Thus, architecture can redistribute where overfitting originates. This is precisely why privacy-aware QML design should consider the encoder together with the model family in which it is deployed.

These findings suggest three design rules for privacy-aware QML. First, feature-map selection should be treated as a security decision, not only an expressivity decision. Encoders such as `Z` and `ZZ` can produce richer quantum feature geometries, but in our experiments, they also more frequently induce train–test asymmetries that support leakage. Second, repeated data encoding should be audited before increasing `reps`. Unlike variational depth, repetitions directly resubmit the input to the circuit, which can degrade utility and privacy simultaneously. Third, architectural inductive bias can act as a structural mitigation. HQNN and QCNN results show that bottlenecks, locality, and convolution-like structure can dampen encoder-induced overfitting even when expressive quantum encoders are used.

10 Limitations

This work presents a controlled structural privacy audit, so the observed hierarchy may not generalize to all QML architectures, datasets, or hardware backends. We use noiseless TorchQuantum simulation deliberately to isolate the effect of the non-trainable encoder U_ϕ, whose feature-map family and repetitions change the induced geometry K_ϕ without changing the trainable ansatz in the way variational depth does. Hardware noise may modulate this hierarchy through gate errors, routing, finite shots, readout noise, and calibration drift; this is complementary to TorchQuantum/QuantumNAS results showing that larger trainable circuits can improve noiseless accuracy but degrade noisy measured accuracy [30]. Broader attacks and direct geometry measurements are left for future work.

11 Conclusion

This paper reframes membership leakage in QML as a structural property of the quantum learning pipeline. Across QNN, HQNN, and QCNN models, we find that the non-trainable encoder can alter the Hilbert-space geometry on which the variational ansatz operates, inducing train–test asymmetries that later appear as membership inference vulnerability. In our controlled regimes, `Z` and `ZZ` encoders and larger feature-map repetitions often produce stronger gaps and stronger attack signals than `effSU2`, while lower-level variational choices such as variational circuit depth, entanglement topology and two-qubit gate type play a secondary role. The practical implication is that privacy-aware QML design should begin at the encoding layer where feature maps and repeated encoding should be audited as security-relevant architectural choices, not only as expressivity knobs.

References

[1] Devansh Arpit, Stanisław Jastrzębski, Nicolas Ballas, David Krueger, Emmanuel Bengio, Maxinder S Kanwal, Tegan Maharaj, Asja Fischer, Aaron Courville, Yoshua Bengio, et al. A closer look at memorization in deep networks. In *International Conference on Machine Learning (ICML)*, pages 233–242. PMLR, 2017.

[2] Marcello Benedetti, Erika Lloyd, Stefan Sack, and Mattia Fiorentini. Parameterized quantum circuits as machine learning models. *Quantum Science and Technology*, 4(4):043001, 2019. doi: 10.1088/2058-9565/ab4eb5.

[3] Jacob Biamonte, Peter Wittek, Nicola Pancotti, Patrick Rebentrost, Nathan Wiebe, and Seth Lloyd. Quantum machine learning. *Nature*, 549(7671):195–202, 2017. doi: 10.1038/nature23474.

[4] Nicholas Carlini, Chang Liu, Úlfar Erlingsson, Jernej Kos, and Dawn Song. The secret sharer: Evaluating and testing unintended memorization in neural networks. In *28th USENIX Security Symposium*, pages 267–284, 2019.

[5] Nicholas Carlini, Steve Chien, Milad Nasr, Shuang Song, Andreas Terzis, and Florian Tramer. Membership inference attacks from first principles. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 1897–1914. IEEE, 2022.

[6] Matthias C. Caro, Elies Gil-Fuster, Johannes Jakob Meyer, Jens Eisert, and Ryan Sweke. Encoding-dependent generalization bounds for parametrized quantum circuits. *Quantum*, 5: 582, November 2021. ISSN 2521-327X. doi: 10.22331/q-2021-11-17-582. URL `http://dx.doi.org/10.22331/q-2021-11-17-582`.

[7] CartwheelX. QuRiFT: Quantum risk and inference fault-line tracer. `https://github.com/CartwheelX/QuRiFT`, 2026. Code repository. Accessed: 2026-05-06.

[8] Daniel J Egger, Claudio Gambella, Jakub Marecek, Scott McFaddin, Martin Mevissen, Giacomo Nannicini, Andrea Oduro, and Stefan Woerner. Quantum computing for finance: state-of-the-art and future prospects. *IEEE Transactions on Quantum Engineering*, 1:1–24, 2020.

[9] Federal Trade Commission. The capital one data breach: Time to check your credit report. `https://consumer.ftc.gov/consumer-alerts/2019/07/capital-one-data-breach-time-check-your-credit-report`, 2019. Accessed: 2026-04-26.

[10] Vitaly Feldman. Does learning require memorization? a short tale about a long tail. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 954–959, 2020.

[11] Vojtěch Havlíček, Antonio D. Córcoles, Kristan Temme, Aram W. Harrow, Abhinav Kandala, Jerry M. Chow, and Jay M. Gambetta. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567(7747):209–212, March 2019. ISSN 1476-4687. doi: 10.1038/s41586-019-0980-2. URL `http://dx.doi.org/10.1038/s41586-019-0980-2`.

[12] Jamie Heredge, Niraj Kumar, Dylan Herman, Shouvanik Chakrabarti, Romina Yalovetzky, Shree Hari Sureshbabu, Changhao Li, and Marco Pistoia. Characterizing privacy in quantum machine learning. *npj Quantum Information*, 11:80, 2025. doi: 10.1038/s41534-025-01022-z.

[13] IBM Quantum. EfficientSU2: Qiskit circuit library documentation, 2026. URL `https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.circuit.library.EfficientSU2`. IBM Quantum Documentation.

[14] IBM Quantum. PauliFeatureMap: Qiskit circuit library documentation, 2026. URL `https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.circuit.library.PauliFeatureMap`. IBM Quantum Documentation.

[15] IBM Quantum. ZFeatureMap: Qiskit circuit library documentation, 2026. URL `https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.circuit.library.ZFeatureMap`. IBM Quantum Documentation.

[16] IBM Quantum. zz_feature_map: Qiskit circuit library documentation, 2026. URL https://quantum.cloud.ibm.com/docs/api/qiskit/qiskit.circuit.library.zz_feature_map. IBM Quantum Documentation.

[17] IBM Quantum Learning. Data encoding. https://learning.quantum.ibm.com/, 2024. Accessed: 2026-04-26.

[18] Sofiene Jerbi, Lukas J. Fiderer, Hendrik Poulsen Nautrup, Jonas M. Kübler, Hans J. Briegel, and Vedran Dunjko. Quantum machine learning beyond kernel methods. *Nature Communications*, 14:517, 2023. doi: 10.1038/s41467-023-36159-y.

[19] Satwik Kundu and Swaroop Ghosh. Security concerns in quantum machine learning as a service, 2024.

[20] Ryan LaRose and Brian Coyle. Robust data encodings for quantum classifiers. *Physical Review A*, 102(3):032420, 2020. doi: 10.1103/PhysRevA.102.032420.

[21] Kosuke Mitarai, Makoto Negoro, Masahiro Kitagawa, and Keisuke Fujii. Quantum circuit learning. *Physical Review A*, 98(3):032309, 2018. doi: 10.1103/PhysRevA.98.032309.

[22] Adrián Pérez-Salinas, Alba Cervera-Lierta, Elies Gil-Fuster, and José I. Latorre. Data re-uploading for a universal quantum classifier. *Quantum*, 4:226, February 2020. ISSN 2521-327X. doi: 10.22331/q-2020-02-06-226. URL http://dx.doi.org/10.22331/q-2020-02-06-226.

[23] Qiskit Community. Qiskit: An open-source framework for quantum computing, March 2017. URL https://github.com/Qiskit/qiskit.

[24] Maria Schuld and Nathan Killoran. Quantum machine learning in feature Hilbert spaces. *Physical Review Letters*, 122(4):040504, 2019. doi: 10.1103/PhysRevLett.122.040504.

[25] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 3–18. IEEE, 2017. doi: 10.1109/SP.2017.41.

[26] Junjian Su, Runze He, Guanghui Li, Sujuan Qin, Zhimin He, Haozhen Situ, and Fei Gao. From membership-privacy leakage to quantum machine unlearning. *Physical Review Applied*, 25: 044056, 2026. doi: 10.1103/PhysRevApplied.25.044056.

[27] Najeeb Ullah, Muhammad Naveed Aman, and Biplab Sikdar. memia: Multilevel ensemble membership inference attack. *IEEE Transactions on Artificial Intelligence*, 6(1):93–106, 2025. doi: 10.1109/TAI.2024.3445326.

[28] U.S. Department of Health and Human Services. Change healthcare cybersecurity incident frequently asked questions. https://www.hhs.gov/hipaa/for-professionals/special-topics/change-healthcare-cybersecurity-incident-frequently-asked-questions/index.html, 2024. Accessed: 2026-04-26.

[29] U.S. Office of Personnel Management. Agency financial report fiscal year 2015. https://www.opm.gov/about-us/reports-publications/agency-archive/2015-agency-financial-report.pdf, 2015. Accessed: 2026-04-26.

[30] Hanrui Wang, Yongshan Ding, Jiaqi Gu, Zirui Li, Yujun Lin, David Z. Pan, Frederic T. Chong, and Song Han. Quantumnas: Noise-adaptive search for robust quantum circuits. In *The 28th IEEE International Symposium on High-Performance Computer Architecture (HPCA-28)*, 2022.

[31] William M. Watkins, Samuel Yen-Chi Chen, and Shinjae Yoo. Quantum machine learning with differential privacy. *Scientific Reports*, 13:2453, 2023. doi: 10.1038/s41598-022-24082-z.

[32] Xi Wei, Saeed Ghadami, Zhifang Hu, and Shuo Gao. Quantum machine learning for medical image analysis: a survey. *arXiv preprint arXiv:2308.14072*, 2023.

[33] Samuel Yeom, Irene Giacomelli, Matt Fredrikson, and Somesh Jha. Privacy risk in machine learning: Analyzing the connection to overfitting. In *2018 IEEE 31st Computer Security Foundations Symposium (CSF)*, pages 268–282. IEEE, 2018. doi: 10.1109/CSF.2018.00027.

[34] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations (ICLR)*, 2017.

A QuRiFT Implementation Details

A.1 Pipeline overview

QuRiFT is implemented as a TorchQuantum-based profiling pipeline for controlled structural privacy analysis in QML [30]. The framework constructs QML configurations, trains each configuration under a fixed optimization protocol, records utility and overfitting metrics, exports prediction-level statistics, and supports downstream membership inference evaluation. The central design principle is controlled intervention: within each experimental family, the training protocol is kept fixed while structural components of the QML pipeline are varied.

The implementation is built on TorchQuantum primitives, which allow quantum circuits to be represented in the PyTorch execution stack. This is useful for our setting because QuRiFT requires large sweeps over circuit structure, feature-map design, repetitions, circuit width, variational depth, entanglement topology, and measurement choices. In particular, TorchQuantum allows circuit parameters to be optimized using PyTorch autograd and supports batched CPU/GPU execution, enabling repeated training of many QNN configurations under comparable conditions.

For each benchmark, the launcher enumerates the Cartesian product of the selected structural settings. Each job is executed independently, and the pipeline writes both per-run logs and aggregate CSV summaries. The sweep variables include feature-map family, feature-map repetitions, circuit width, variational depth, feature-map entanglement and gate type, and variational-layer entanglement and gate type. Synthetic sweeps use smaller circuit widths and depths, while MNIST sweeps scale the number of wires upward while preserving the same structural axes across model families.

A.2 Simulation backend and hardware scope

All experiments reported in this paper are conducted using controlled noiseless TorchQuantum simulation on GPU-equipped DGX servers. This is a deliberate methodological choice. The central object of study is the non-trainable encoder U_ϕ: changing the feature-map family or its repetitions changes the induced Hilbert–Schmidt geometry K_ϕ, but does not change the trainable ansatz in the same way as increasing variational depth. Noiseless simulation therefore provides the cleanest intervention for testing whether a fixed representation-forming circuit can induce the train–test asymmetry that membership inference attacks exploit.

Hardware noise is an important deployment-level factor, but it answers a different question from the one studied here. In a noisy backend, measured utility and privacy behavior would also depend on gate errors, finite-shot variability, qubit mapping, routing overhead, readout noise, and calibration drift. Prior TorchQuantum/QuantumNAS results show that increasing trainable circuit capacity can improve noiseless accuracy, while additional gates may accumulate hardware noise and eventually reduce noisy measured accuracy [30]. Our study isolates a complementary mechanism: encoder-induced representation geometry as a source of overfitting-driven membership leakage. A hardware-aware extension of QuRiFT should therefore study how backend noise modulates this encoder-induced privacy hierarchy, rather than treating noisy execution as necessary for establishing the structural mechanism itself.

A.3 Angle encoding and input normalization

All vector-valued inputs in QuRiFT are encoded through rotation gates. After preprocessing, each scalar feature is normalized to a bounded angular range and inserted as the angle of a single-qubit rotation. In the simplest Y-axis form,

$$R_Y(x_k)|0\rangle = \cos\Big(\frac{x_k}{2}\Big)\,|0\rangle + \sin\Big(\frac{x_k}{2}\Big)\,|1\rangle.$$

For an input vector $x = (x_1, \ldots, x_D)$, feature values are assigned to the available encoding gates through a pre-built tiled feature-map operator list. The tiling is determined by the input dimension D, circuit width `n_wires`, feature-map family `fm_kind`, repetition count `reps`, and padding mode `pad_mode`. If the number of features exceeds what can be placed in one encoding tile, the encoder extends the same feature-map pattern across repeated/tiled blocks until the input features are consumed. If the final tile contains more encoding gates than remaining features, the unused gates are filled

according to the selected padding rule, such as zero padding, wrapping, or repeating the last available feature. Thus, the implementation should be understood as tiled encoding with padding, rather than as dynamically creating new encoder layers during execution. For MNIST, each architecture first produces a compact 1×16 representation before this tiled encoding step: dense QNN uses PCA, bilinear pooling, or average pooling; HQNN uses a trainable CNN preprocessor; and QCNN uses a local quantum-filter/quanvolutional preprocessor, as detailed in Appendix A.5.

The feature-map family determines how these angle-loaded values are used. The `Z` map applies feature-dependent single-qubit phase rotations; the `ZZ` map additionally introduces data-dependent two-qubit phase interactions; and `effSU2` combines angle-based rotations with hardware-efficient entangling layers. Feature-map repetitions increase the number of tiled data-dependent encoding blocks, thereby enriching the representation without increasing trainable parameters in the same way as variational-depth scaling. In our implementation, this tiling interacts with the input dimension and padding mode: features are consumed across repeated encoding tiles, and any unused gates in the final tile are filled using the selected padding policy.

We normalize inputs before angle encoding because rotation gates are periodic: two values separated by multiples of 2π may correspond to the same physical rotation. This avoids trivial aliasing and keeps comparisons across feature-map families controlled. Angle encoding is used because it is a standard near-term choice for continuous-valued QML inputs, yields shallow hardware-compatible circuits, supports data re-uploading, and avoids the arbitrary state-preparation overhead associated with amplitude encoding [17, 19–22].

A.4 Feature-map templates

The feature-map templates used in QuRiFT follow standard Qiskit circuit-library constructions, but are implemented through TorchQuantum-compatible operation lists so that the same templates can be trained and swept efficiently under the PyTorch/TorchQuantum backend. In particular, our `Z`-feature-map implementation follows the Qiskit `Z`-feature-map template, which is a Pauli feature map with Pauli strings fixed to `Z` and does not introduce entangling gates by default [15, 23]. The `ZZ` feature map follows the corresponding single- and two-qubit Pauli-`Z` evolution structure [16, 23]. The more general Pauli feature map follows Qiskit's PauliFeatureMap abstraction over user-specified Pauli strings, repetitions, and entanglement choices [14, 23]. When efficient-SU(2)-style encoders or ansatz blocks are used, they follow the standard layered pattern of single-qubit SU(2) rotations and entangling operations used in Qiskit's EfficientSU2 family [13].

This implementation choice ensures that the encoder families studied in the paper correspond to standard feature-map designs, while still allowing the rest of the pipeline to remain differentiable, batched, and compatible with TorchQuantum-based training.

A.5 Benchmarks and preprocessing

We use two benchmark regimes. The synthetic regime includes *Moons*, *Circles*, and *Blobs*. The launcher uses fixed train/validation/test splits for each synthetic task. The data-generation parameters are fixed across sweeps so that observed differences in generalization gap can be attributed to structural changes in the QML pipeline rather than to dataset resampling. In particular, *Moons* and *Circles* use noise-controlled nonlinear decision boundaries, while *Blobs* provides a clustered binary classification task. For *Moons*, polynomial feature expansion is enabled in the synthetic pipeline. All vector inputs are scaled consistently before quantum encoding.

The image regime uses a four-class MNIST subset over digits $\{0, 1, 3, 8\}$. To keep the quantum input dimension fixed across model families, each MNIST sample is converted to a compact 1×16 representation before the main quantum encoder. The compression mechanism depends on the architecture family. For the dense QNN, the 28×28 image is reduced to 16 features using a non-trainable preprocessing choice, such as PCA, bilinear pooling, or average pooling. For the HQNN, a trainable CNN preprocessor maps the image to a 16-dimensional latent vector; this hybrid design is motivated by prior QNN membership-leakage and quantum machine-unlearning work [26]. For the QCNN, local quantum-filter/quanvolutional operations, implemented using TorchQuantum-compatible primitives, process image patches and produce a 16-dimensional representation before the downstream encoder and PQC stack [30]. In HQNN and QCNN, the measured quantum features are further passed through an MLP post-processing head before final classification. This design keeps

the quantum input size comparable while allowing each architecture to express its intended inductive bias.

After the 1×16 representation is obtained, features are passed to the tiled feature-map encoder; the operator list is constructed from D, `n_wires`, `reps`, `fm_kind`, and `pad_mode`, with leftover gates filled by the selected padding rule. For the default `eff_su2` setting, `pad_mode` is `repeatlast`, so leftover gates reuse the last available feature rather than dropping the input.

A.6 Model families

We evaluate three QML model families and one optional classical baseline.

QNN. The dense QNN is the fully variational baseline. For MNIST, the original 28×28 image is first compressed to a 1×16 feature vector using a non-trainable preprocessing method, selected from PCA, bilinear pooling, or average pooling. This compact vector is then passed to the quantum feature map $U_\phi(x)$, followed by the trainable variational circuit U_θ, Pauli-Z measurement, and a final classical classifier. The quantum operations in this baseline are constructed using TorchQuantum-compatible circuit primitives [30]. This model provides the cleanest test of encoder-induced overfitting because the image compression step is not itself a trainable feature extractor.

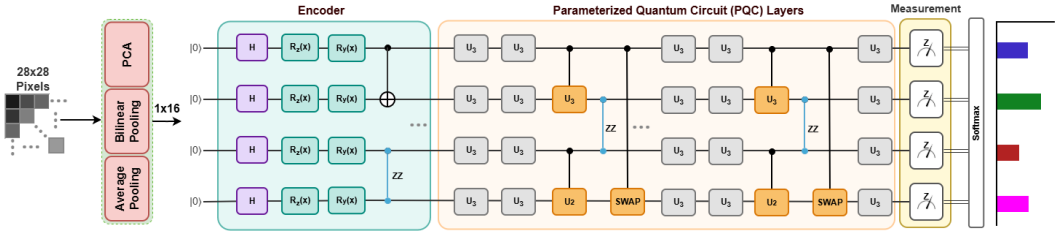


Figure 9: Circuit-level dense QNN model used in QuRiFT. The MNIST input is compressed from 28×28 pixels to a 1×16 feature vector using PCA, bilinear pooling, or average pooling. The resulting vector is encoded through $U_\phi(x)$, processed by parameterized quantum circuit (PQC) layers U_θ, measured using Pauli-Z observables, and mapped to class probabilities through a classical softmax head. The quantum gates, entangling operations, and measurements are implemented using TorchQuantum-compatible primitives [30].

HQNN. The HQNN augments the quantum stack with a trainable classical CNN preprocessor. Instead of directly compressing the image through PCA or pooling, the CNN maps the 28×28 input to a 1×16 latent vector. This vector is passed to the quantum feature map $U_\phi(x)$, followed by the variational circuit U_θ and Pauli-Z measurement. The measured quantum features are then post-processed by a classical MLP head before the final softmax classification. We include this model family to test whether a trainable classical bottleneck, together with post-measurement classical processing, can dampen the direct influence of aggressive quantum feature maps and reduce privacy-relevant overfitting. The use of a hybrid CNN–QNN structure is motivated by prior QNN membership-leakage and quantum machine-unlearning studies [26].

QCNN. The QCNN uses a local quantum-filter, or quanvolutional, preprocessing stage as an analogue of a convolutional filter. Local image patches are encoded into small quantum circuits, processed by local variational blocks, and measured through Pauli observables. The resulting spatial feature map is flattened or pooled to produce a 1×16 representation before the main downstream quantum/classical stack. The compact representation is then encoded, processed by PQC layers, measured, and passed through a classical MLP post-processing head before final softmax classification. The quantum-filter and local PQC operations are implemented using TorchQuantum-compatible primitives [30]. This architecture imposes locality and provides an inductive-bias comparison against the dense QNN.

Classical baseline. The modular implementation also supports a classical MLP-style baseline for comparison. This wrapper is not the focus of the main paper, but it is useful for sanity-checking utility trends and comparing quantum structural effects against a non-quantum model family.

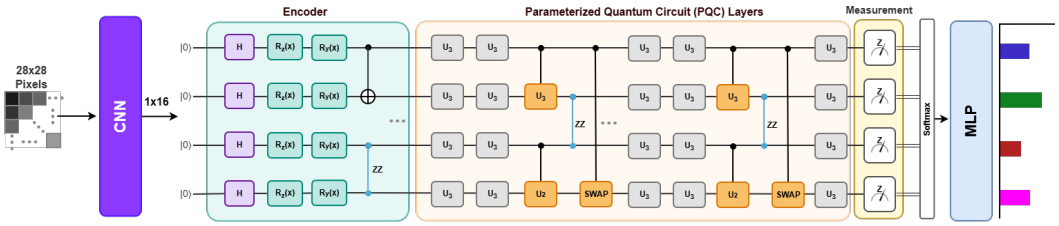


Figure 10: Circuit-level HQNN model used in QuRiFT. The MNIST image is first processed by a trainable classical CNN preprocessor, which outputs a 1×16 latent representation. This representation is encoded through $U_\phi(x)$, transformed by PQC layers U_θ, and measured through Pauli-Z observables. The measured quantum features are then passed through a classical MLP post-processing head before final softmax classification. The hybrid CNN–QNN structure is motivated by prior QNN membership-leakage and quantum machine-unlearning work [26].

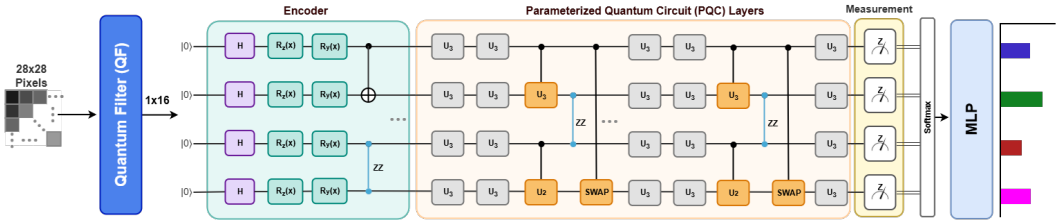


Figure 11: Circuit-level QCNN/quantum-filter model used in QuRiFT. The MNIST image is first processed through local quantum-filter operations over image patches, producing a compact 1×16 representation. This representation is then passed through the encoder $U_\phi(x)$, transformed by PQC layers U_θ, and measured using Pauli-Z observables. The measured quantum features are post-processed by a classical MLP head before final softmax classification. The quantum-filter and PQC components are implemented using TorchQuantum-compatible primitives [30].

A.7 Structural sweep variables

The main sweep variables are:

`model_type`, `fm_kind`, `reps`, `n_wires`, `depth`, `fm_ent`, `ql_ent`, `fm_op`, `ql_op`.

Here, `model_type` selects the architecture wrapper, such as QNN, HQNN, QCNN, or a classical baseline. The feature-map family `fm_kind` includes `Z`, `ZZ`, `pauli`, and `effSU2`. The variable `reps` controls the number of repeated data-encoding blocks. Circuit width is controlled by `n_wires`, and variational ansatz depth by `depth`. The variables `fm_ent` and `ql_ent` specify feature-map and ansatz entanglement topologies, while `fm_op` and `ql_op` specify the two-qubit gate families used in the encoder and variational circuit.

The key comparison in the paper is between feature-map repetitions and variational depth. Both increase circuit complexity, but they do so in different parts of the pipeline. Repetitions repeatedly inject input-dependent structure into the quantum feature state, whereas depth primarily increases the expressivity of the trainable variational ansatz. This distinction allows QuRiFT to test whether privacy risk originates more strongly from data-dependent encoding or from trainable circuit capacity.

A.8 Training and evaluation

All models are trained end-to-end using Adam optimization, cosine-annealing learning-rate scheduling, and negative log-likelihood loss on log-softmax outputs. The training protocol is fixed within each experimental family so that comparisons across feature maps, repetitions, widths, depths, and variational-layer choices reflect structural effects rather than optimizer changes. For each configuration, QuRiFT records train, validation, and held-out test accuracy and loss.

For membership inference analysis, the pipeline exports prediction vectors and derived statistics for each sample, including loss, entropy, confidence, margin, and correctness. These outputs provide a consistent attack input across datasets, architectures, and structural regimes.

A.9 Membership inference protocol

Membership inference is performed on selected baseline, stress, and hard configurations. The threat model is black-box: the adversary can query the trained target QML model and observe the full prediction vector $\mathbf{p}(x) = \mathcal{F}_{\text{target}}(x)$, but cannot inspect the model parameters, gradients, quantum states, circuit internals, training data, or optimizer trajectory. This follows the output-based membership inference setting commonly used in classical MIAs [5, 25, 33] and is consistent with the QML membership-leakage oracle setting considered in quantum machine-unlearning work [26].

For attack evaluation, we query the trained target QML model on examples whose membership status is known to the evaluator: training samples are treated as members and held-out test samples as non-members. The prediction vector is then converted into attack features, including confidence, entropy, margin, loss, and correctness when the candidate label is available. The attack model learns to distinguish member from non-member outputs, and we report attack AUC and fixed-FPR true-positive rates. The attack evaluation is used as validation of the structural proxy: configurations with larger train–test gaps should, in general, produce stronger membership signals, although the gap need not fully determine attack success.

The three structural regimes are defined as follows. The *baseline* regime is a reference configuration and provides the comparison point for attack strength. The *stress* regime is selected to exhibit stronger structural overfitting, typically through a larger train–test gap induced by encoder choice, repetitions, width, or related circuit-scaling choices. The *hard* regime retains stronger held-out utility and a smaller or more controlled gap, making the member/non-member distinction harder for the attack. These regimes are not new training objectives; they are post-hoc selections from the controlled sweep used to test whether structural overfitting translates into measurable leakage. For the MNIST architecture comparison, the regimes are matched across QNN, HQNN, and QCNN: within each regime, `fm_kind`, `reps`, `n_wires`, `depth`, `ql_ent`, and `ql_op` are fixed, and only the architecture wrapper and its front-end/head differ.

B QNN Architecture and Privacy-Analysis Flow

Figure 12 summarizes the QNN architecture and privacy-analysis flow used in QuRiFT. The upper region corresponds to quantum processing, where the input is encoded by $U_\phi(x)$, transformed by the variational circuit U_θ, and measured through Pauli observables. The lower region corresponds to classical processing, including data handling, loss computation, optimization, post-processing, and privacy evaluation. The red pathway highlights the mechanism studied in this work: train/test behavioral differences induce the generalization gap $\Delta_{\text{gen}} = \text{Acc}_{\text{train}} - \text{Acc}_{\text{test}}$, which is used as a structural proxy for membership-inference risk.

C Modular Organization and Implementation of QuRiFT

Figure 13 summarizes the modular software organization of QuRiFT. The purpose of this figure is not to introduce an additional empirical result, but to clarify how the controlled sweeps reported in the main paper are instantiated in a reproducible and extensible way. QuRiFT is designed around a fixed evaluation flow while keeping the major QML design choices replaceable. This allows the same experimental protocol to be applied across different datasets, preprocessing routines, feature maps, circuit widths, variational depths, entanglement topologies, measurement heads, architecture wrappers, and privacy metrics.

The left column of Fig. 13 shows the core execution pipeline. A dataset module first produces labeled samples (x, y), which are passed through preprocessing. The processed input is then passed to the quantum feature-map module $U_\phi(x)$, which implements the classical-to-quantum encoding analyzed in the main paper. This separation is deliberate: the feature map is not treated as a fixed background component, but as a first-class structural variable whose family, repetitions, padding policy, and entanglement structure can be varied systematically.

After encoding, the variational circuit module U_θ applies the trainable quantum layer. This module contains ansatz-level choices such as circuit depth, two-qubit gate family, and entanglement topology. The measurement and head module maps the quantum state back to classical outputs through Pauli observables, joint-Pauli measurements, pooling rules, and a final classifier head. The resulting

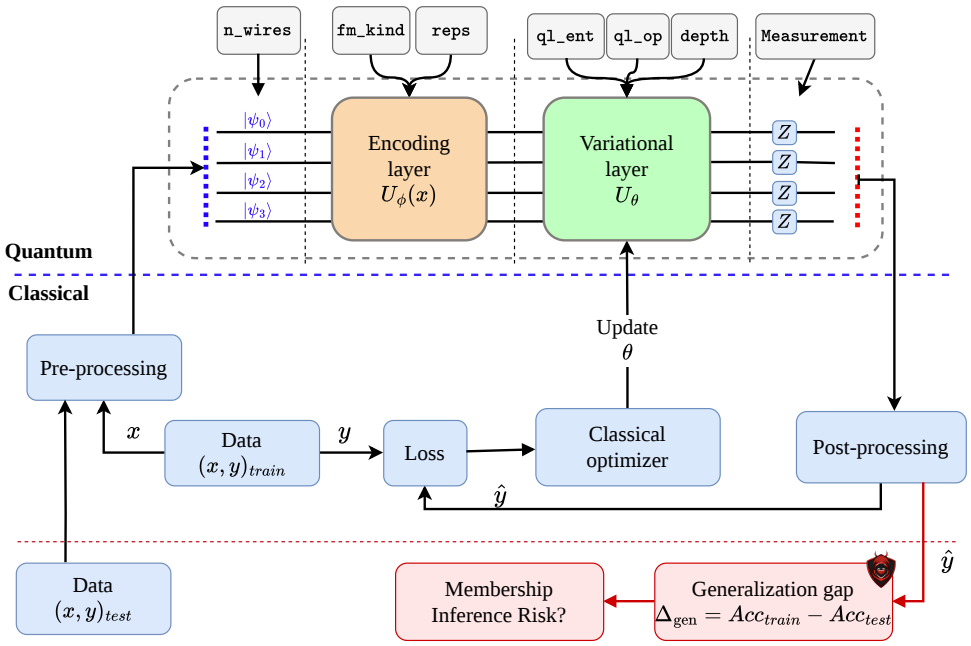


Figure 12: Architecture and privacy-analysis flow of the QNN pipeline studied in this work. Classical inputs are pre-processed and encoded into quantum states through the feature map $U_\phi(x)$, transformed by the trainable variational circuit U_θ, measured, and post-processed to obtain predictions. Structural knobs such as `fm_kind`, `reps`, `n_wires`, `depth`, `ql_ent`, and `ql_op` are audited for their effects on overfitting and membership leakage.

logits or probabilities are used for both target-model utility evaluation and membership-inference analysis. In the final stage, QuRiFT records the train–test gap Δ_{gen}, attack AUC, fixed-FPR TPR, and factor-attribution statistics.

The surrounding panels indicate the replaceable module families exposed by the framework. Dataset modules include MNIST, synthetic datasets such as Moons, Circles, and Blobs, and custom vector datasets. Feature-map modules include `Z`, `ZZ`, Pauli, and efficient-SU(2)-style encoders. Variational and measurement modules include different variational blocks, repeated circuit templates, random layers, entanglement patterns, two-qubit gates, Pauli measurement rules, and classical heads. The dashed extension boxes indicate that these components are not hard-coded: a new dataset, encoder, ansatz, measurement rule, architecture wrapper, or privacy metric can be added without rewriting the full evaluation pipeline.

The architecture-wrapper layer should be read as an upstream configuration choice rather than as an output of the model. In a given run, the user or sweep engine selects a wrapper such as QNN, HQNN, QCNN, or a classical baseline. That wrapper determines how the modular components are assembled into an executable model. For example, a standard QNN uses the feature map and variational circuit directly; an HQNN inserts a hybrid classical–quantum interface; and a QCNN imposes a more structured architecture with convolution-like inductive bias. This is why Fig. 13 connects the architecture wrappers to the sweep engine rather than to the model-output box: model-family selection occurs before training and evaluation, not after prediction.

The sweep orchestrator at the bottom generates controlled configurations and launches the corresponding runs. In the experiments used in this paper, the sweep varies structural parameters such as `model_type`, `fm_kind`, `reps`, `n_wires`, `depth`, `ql_ent`, and `ql_op`. Each configuration is trained under a fixed protocol, and the resulting logs, CSV records, plots, and privacy-analysis outputs are stored for downstream aggregation. This design enables controlled comparison: instead of changing many implementation details at once, QuRiFT changes one or more structural factors while keeping the evaluation flow consistent.

This modular organization also explains how QuRiFT can be extended beyond the experiments reported in the paper. A future user can add a new Qiskit-style feature map, a hardware-aware ansatz, a noise model, a label-only membership inference attack, a query-limited attack, or a new factor-attribution metric by implementing the corresponding module and registering it in the sweep configuration. The rest of the pipeline remains unchanged. Thus, QuRiFT is intended not only as the implementation used for this study, but also as a reusable audit framework for testing how QML architecture choices affect utility, overfitting, and membership-inference risk.

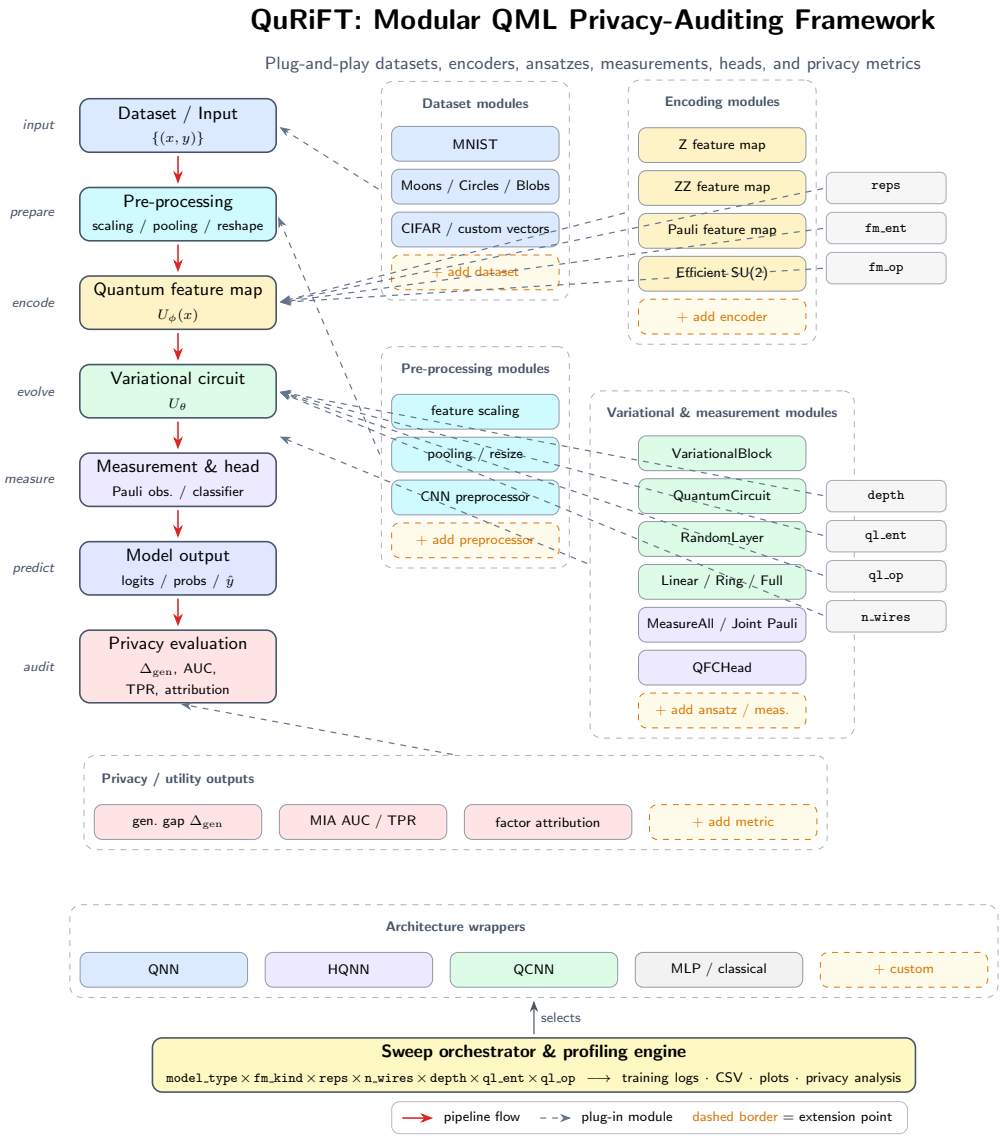


Figure 13: Modular organization of QuRiFT. The left column shows the core QML evaluation flow from dataset construction to privacy analysis. The surrounding panels expose replaceable module families, including datasets, preprocessing routines, Qiskit-style feature-map templates, TorchQuantum-based variational circuits, measurement heads, architecture wrappers, and privacy metrics. Architecture wrappers such as QNN, HQNN, QCNN, and classical baselines are selected upstream by the sweep configuration; they determine how the modular components are assembled into an executable model. The sweep orchestrator then varies structural parameters, trains the selected configuration, and records utility and privacy outputs such as Δ_{gen}, attack AUC, fixed-FPR TPR, plots, logs, and factor-attribution statistics.

D Encoder-Induced Representation and Privacy Risk

The role of the encoder can be made explicit by viewing it as the first representation-forming stage of the QML pipeline. Given a classical input x, the feature map $U_\phi(x)$ maps the initial state ρ_0 to

$$\rho_\phi(x) = U_\phi(x)\rho_0 U_\phi(x)^\dagger.$$

This mapping induces the Hilbert–Schmidt kernel

$$K_\phi(x_i, x_j) = \mathrm{Tr}(\rho_\phi(x_i)\rho_\phi(x_j)),$$

which determines how pairs of inputs are arranged in the quantum feature space before the variational circuit is trained. Thus, changing the encoder changes the geometry on which the ansatz operates, even when the optimizer, loss, and trainable circuit family are held fixed.

This geometry matters for privacy because it can affect how easily the downstream trainable circuit separates training samples from unseen samples. Some encoders may concentrate feature states, others may spread class-conditioned states apart, and others may make the encoded state more sensitive to small changes in the input. These effects are not membership leakage by themselves. Rather, they shape the conditions under which training-specific structure becomes easier to fit, increasing the train–test gap Δ_{gen}. Once such a gap appears, it can be reflected in the model's output probabilities, which are precisely the signals exploited by membership inference attacks.

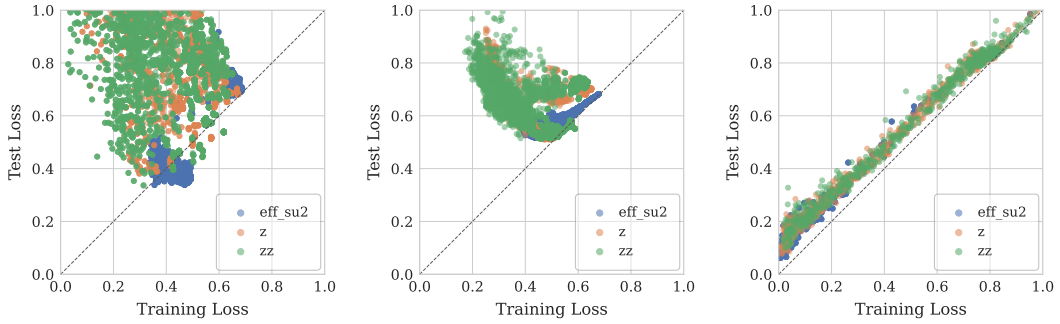
We therefore interpret membership leakage as depending not only on the final trained predictor, but also on the representation geometry induced upstream by the encoder. Informally,

$$\mathcal{L}_{\mathrm{MIA}} \approx f(\Delta_{\mathrm{gen}}, K_\phi, S_\phi, C_\phi),$$

where K_ϕ captures feature-state similarity, S_ϕ denotes local encoding sensitivity, and C_ϕ denotes class-conditioned separation in the induced quantum feature space. This expression is not intended as a closed-form leakage theorem. It is a useful structural view: the encoder shapes the geometry, the geometry can condition overfitting, and overfitting can become visible to an output-based adversary through prediction-vector behavior. Recent QML privacy work shows that QNN outputs can indeed leak membership information and has studied quantum machine unlearning as a post-training mitigation [26]; our focus is complementary, asking which upstream design choices make such leakage more likely to arise in the first place.

E Additional Encoder-Induced Overfitting Results

Figure 14 reports the encoder-induced overfitting panels omitted from the main paper. These results complement Fig. 1 by showing that the encoder-dependent train–test loss behavior extends across additional synthetic datasets and the HQNN architecture.

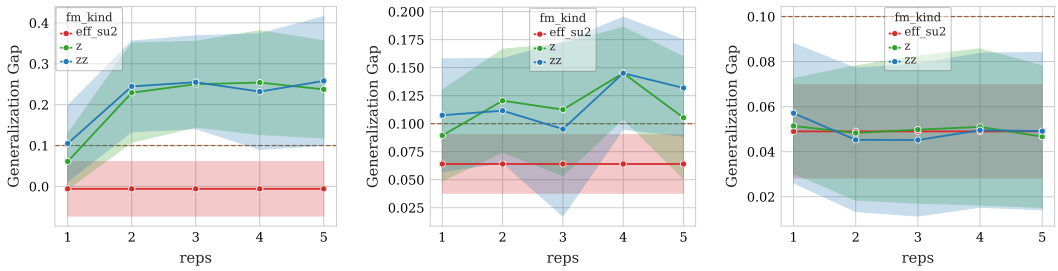


(a) Blobs–QNN (b) Circles–QNN (c) MNIST–HQNN

Figure 14: Additional train–test loss scatter plots for encoder-induced overfitting. Together with Fig. 1, these panels show that feature-map choice affects overfitting behavior across different data geometries and architectures.

F Additional Repetition-Sensitivity Results

Figure 15 reports the repetition-sensitivity panels omitted from the main paper. These results complement Fig. 2 by showing that the effect of feature-map repetitions extends across additional synthetic datasets and the HQNN architecture.

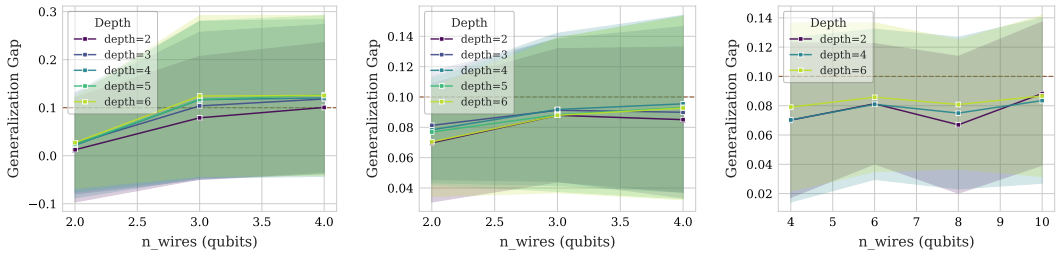


(a) Blobs (b) Circles (c) MNIST–HQNN

Figure 15: Additional generalization-gap trends as a function of feature-map repetitions. Together with Fig. 2, these panels show that repeated data encoding amplifies overfitting for more aggressive feature maps, while the dependence is weaker for more stable encodings and for HQNN.

G Additional Width–Depth Interaction Results

Figure 16 reports the width–depth interaction panels omitted from the main paper. These results complement Fig. 3 and show that the same qualitative interaction extends across additional synthetic settings and MNIST–QCNN.



(a) Blobs (b) Circles (c) MNIST–QCNN

Figure 16: Additional generalization-gap trends as a function of circuit width across depths. Together with Fig. 3, these panels support the conclusion that width can amplify overfitting more strongly in deeper circuits, although the strength of the effect varies with dataset and architecture.

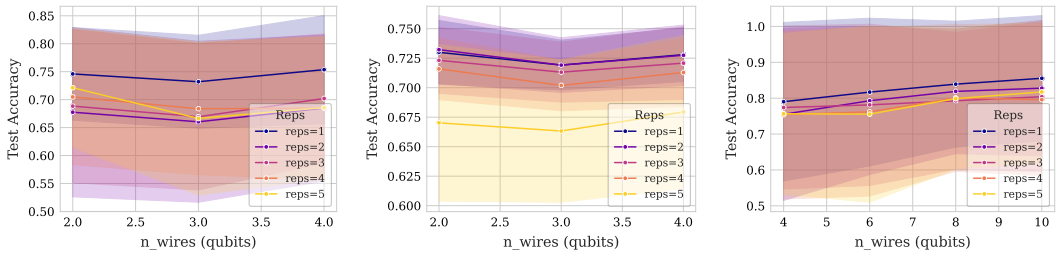
H Additional Utility Results

Figure 17 reports additional utility results for circuit width and feature-map repetitions. These panels complement the main-paper discussion and further support the finding that repetitions consistently degrade test accuracy, while width has a dataset- and architecture-dependent effect.

Figure 18 reports additional depth-sensitivity results across synthetic and MNIST settings. These panels show that variational depth provides weaker and less reliable utility gains than feature-map repetitions or circuit width.

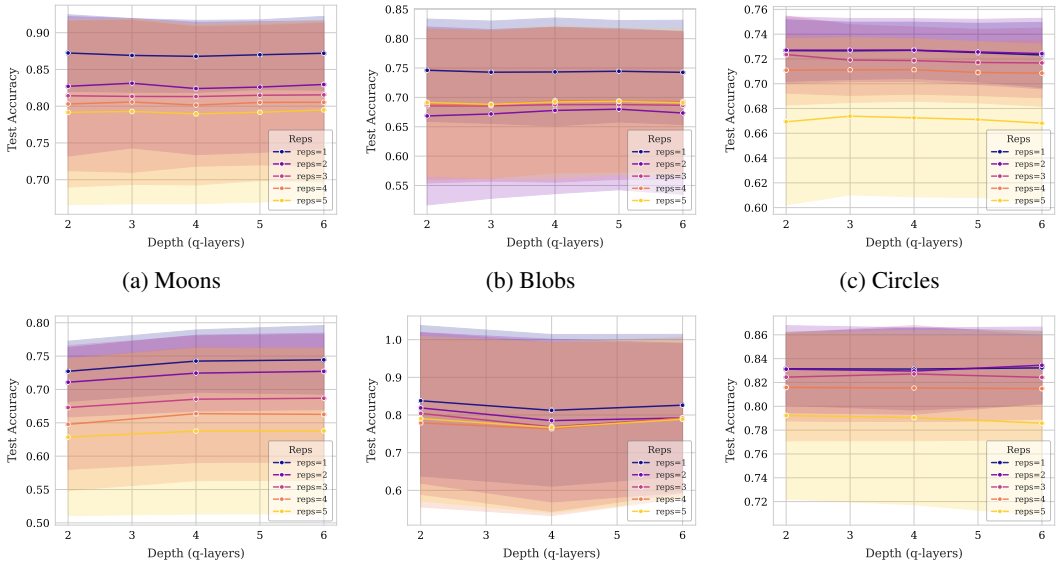
I Entanglement and Gate-Type Interactions

Figure 19 and Table 1 report additional evidence for the secondary role of variational-layer entanglement topology and two-qubit gate choice. The figure complements the main-paper violin plot, while



(a) Blobs (b) Circles (c) MNIST–HQNN

Figure 17: Additional test-accuracy trends as a function of circuit width across feature-map repetitions. Together with the main-paper results, these panels reinforce the conclusion that repeated data encoding is a consistent source of utility degradation, while the effect of width varies across datasets and architectures.



(a) Moons (b) Blobs (c) Circles

(d) MNIST–QNN (e) MNIST–HQNN (f) MNIST–QCNN

Figure 18: Test accuracy as a function of variational depth across feature-map repetitions. Across these settings, depth yields smaller and less consistent utility changes than feature-map repetitions, supporting the main-paper conclusion that repeated data encoding is the more privacy-relevant scaling knob.

the table gives detailed FM–QL interaction values on Circles. Overall, the gap values remain within a relatively narrow range across most entanglement–gate pairings, reinforcing the conclusion that these lower-level variational-layer choices play a secondary role relative to encoder family, repetitions, and width.

J Synthetic Stress and Hard Regimes

Figure 20 shows baseline, stress, and hard configurations for QNNs on the synthetic datasets. These results complement the MNIST architecture comparison in the main paper by showing that even within a fixed QNN family, structural choices can move the model between overfitted and better-generalizing regimes.

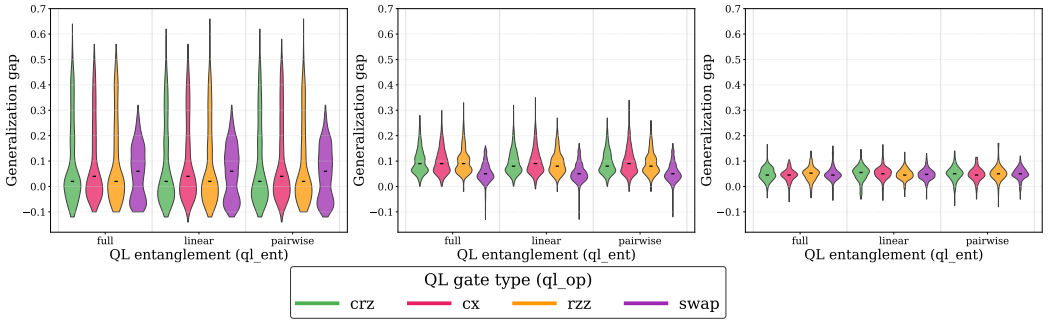
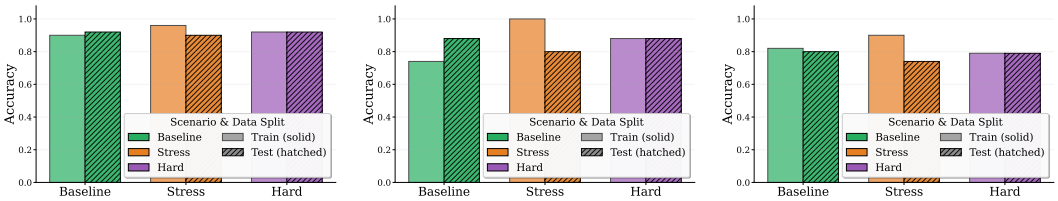


Figure 19: Additional interaction between variational-layer entanglement topology and two-qubit gate type on Blobs–QNN, Circles–QNN, and MNIST–HQNN from left to right. The generalization-gap distributions remain broadly similar across `ql_ent` choices and `ql_op` choices, indicating that these lower-level circuit choices introduce modest variation compared with feature-map design, repetitions, and circuit width.

QL (`ql_ent`, `ql_op`)	FM full		FM linear		FM ring	
	cx	cz	cx	cz	cx	cz
full, crz	0.075	0.065	0.070	0.068	0.075	0.069
full, cx	0.072	0.067	0.069	0.067	0.072	0.066
full, rzz	0.075	0.067	0.076	0.067	0.072	0.070
full, swap	0.058	0.034	0.064	0.033	0.084	0.037
linear, crz	0.075	0.060	0.073	0.063	0.075	0.066
linear, cx	0.071	0.068	0.065	0.070	0.070	0.066
linear, rzz	0.069	0.063	0.073	0.062	0.069	0.063
linear, swap	0.059	0.032	0.062	0.034	0.080	0.037
pairwise, crz	0.070	0.062	0.073	0.064	0.074	0.065
pairwise, cx	0.067	0.069	0.066	0.068	0.067	0.066
pairwise, rzz	0.067	0.061	0.073	0.062	0.069	0.063
pairwise, swap	0.060	0.030	0.063	0.033	0.081	0.035

Table 1: FM–QL structural interaction on Circles. Each cell reports the generalization gap for a given combination of QL entanglement/gate pair (`ql_ent`, `ql_op`) and feature-map topology/gate pair (`fm_ent`, `fm_op`). Light shading marks low-gap combinations; darker shading marks the highest-gap pocket.

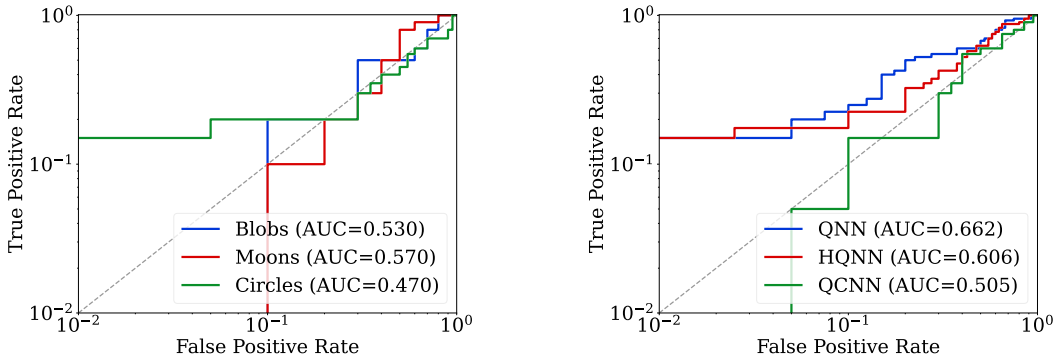


(a) Moons (b) Blobs (c) Circles

Figure 20: Baseline, stress, and hard configurations for synthetic QNNs. Stress configurations tend to achieve higher training accuracy but larger train–test gaps, while hard configurations preserve stronger test accuracy with smaller gaps.

K Additional Membership Inference ROC Curves

The main paper reports stress and hard regimes because they provide the clearest test of the structural privacy hypothesis. Figure 21 provides the corresponding baseline ROC curves for completeness.



(a) Synthetic QNN–baseline

(b) MNIST–baseline

Figure 21: Baseline log-scale ROC curves for membership inference attacks. These curves provide reference behavior for the stress and hard regimes reported in Fig. 7.

The baseline ROC curves in Fig. 21 provide an intermediate reference point between the stress and hard regimes shown in the main paper. They confirm that the stronger attack performance observed under stress configurations is not merely a consequence of dataset choice or attack implementation, but is tied to structurally induced memorization. Conversely, the hard configurations reduce attack advantage because their design choices suppress the train–test behavioral discrepancy exploited by membership inference attacks.

L Structural Regimes and Membership Inference Metrics

Table 2 reports the complete target-model and membership inference metrics for the baseline, stress, and hard structural regimes used in our MI attack evaluation. For each dataset–architecture pair, we report target train accuracy, target test accuracy, attack accuracy, true positive rate at $\mathrm{FPR} = 0.1$, and ROC AUC.

The full attack metrics show that structurally selected stress configurations often, but not always, produce the strongest membership inference signal. Stress has the highest AUC for MNIST–QNN, MNIST–QCNN, Blobs–QNN, and Circles–QNN, with the clearest separation on Blobs where the attack AUC rises to 0.870. However, MNIST–HQNN and Moons–QNN do not follow this ordering: the HQNN hard configuration yields the highest AUC, while the Moons baseline has the largest AUC despite a small train–test gap. These exceptions are informative. They show that the generalization gap is a useful privacy-risk proxy, but not a complete predictor of attack success. MI attack performance also depends on the distributional geometry of the model outputs, class-confidence structure, and the separability of member and non-member posterior vectors.

M Membership Inference Sensitivity to Depth and Repetitions

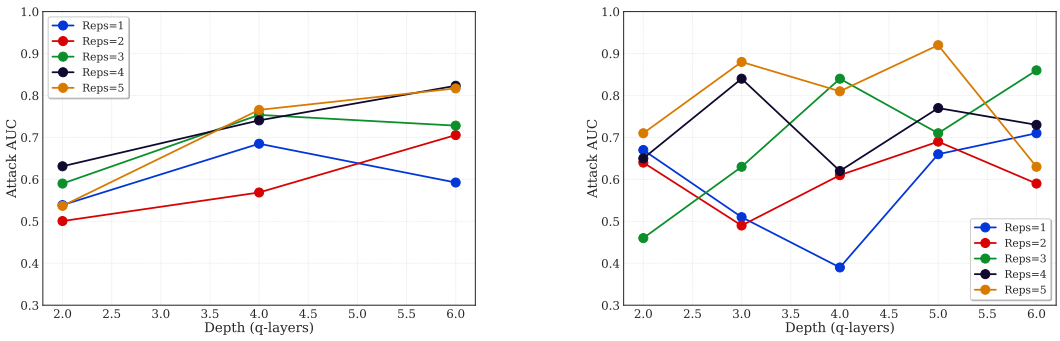
Figure 22 reports additional MIA results isolating variational depth and feature-map repetitions. These results are consistent with the hierarchy observed in the main paper: repetitions produce clearer changes in attack behavior, while depth alone has a weaker and less monotonic effect.

N Correlation Matrix for Structural Privacy Metrics

The main paper reports the gap–AUC scatter plot and the compact correlation table in Fig. 4. For completeness, Fig. 23 reports the full correlation matrix linking structural variables, utility metrics, and attack metrics.

Configuration		Target train acc	Target test acc	Attack acc	TPR @ FPR $= 0.1$	Attack AUC
MNIST–QNN	Baseline	0.940	0.825	0.637	0.250	0.662
	Stress	1.000	0.745	0.600	0.500	**0.674**
	Hard	0.760	0.765	0.550	0.200	0.601
MNIST–HQNN	Baseline	0.985	0.935	0.575	0.225	0.606
	Stress	1.000	0.945	0.562	0.375	0.629
	Hard	1.000	0.910	0.625	0.425	**0.681**
MNIST–QCNN	Baseline	0.890	0.860	0.525	0.150	0.505
	Stress	0.990	0.840	0.575	0.200	**0.647**
	Hard	0.850	0.860	0.550	0.100	0.507
Blobs–QNN	Baseline	0.740	0.880	0.550	0.200	0.530
	Stress	1.000	0.800	0.800	0.700	**0.870**
	Hard	0.880	0.880	0.700	0.500	0.750
Moons–QNN	Baseline	0.900	0.920	0.450	0.100	**0.570**
	Stress	0.960	0.900	0.600	0.100	0.470
	Hard	0.920	0.920	0.450	0.200	0.470
Circles–QNN	Baseline	0.820	0.800	0.450	0.200	0.470
	Stress	0.900	0.740	0.550	0.200	**0.578**
	Hard	0.790	0.790	0.500	0.050	0.440

Table 2: Target performance and membership inference attack strength for baseline, stress, and hard structural regimes. For each dataset–architecture pair, we report target train/test accuracy, attack accuracy, true positive rate at $\mathrm{FPR} = 0.1$, and ROC AUC. Bold values mark the configuration with the highest attack AUC within each block.



(a) MNIST (b) Moons

Figure 22: Effect of variational depth and feature-map repetitions on membership-inference risk for MNIST and Moons.

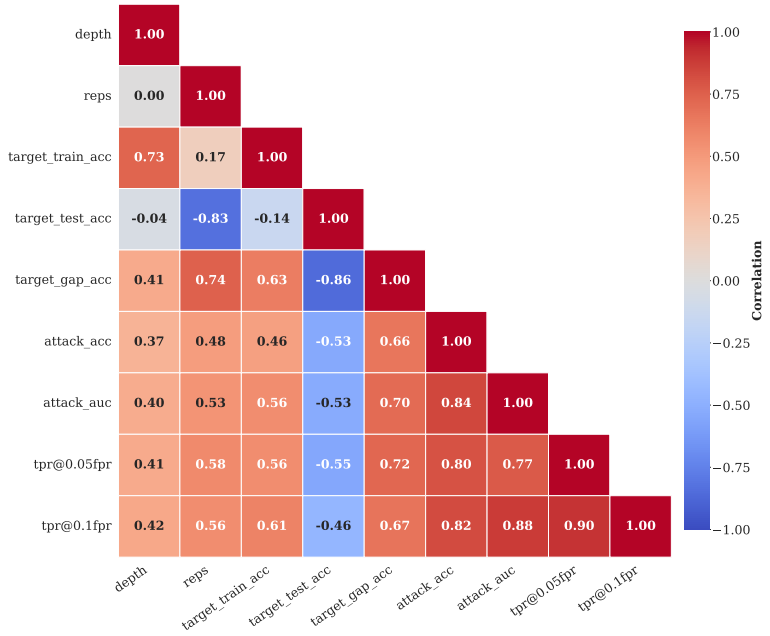
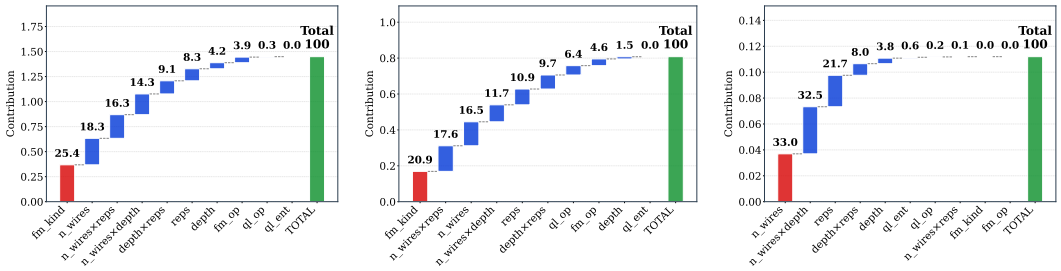


Figure 23: Full correlation matrix for structural privacy analysis. The matrix links architecture and encoding variables to utility and attack metrics, complementing the compact correlation table and gap–AUC scatter plot in Fig. 4.

O Additional Factor-Attribution Results

Figure 24 reports the factor-attribution panels omitted from the main paper. These results complement Fig. 8 by showing that the main attribution hierarchy is not an artifact of the selected panels. Across the additional settings, the largest contributions to the generalization gap come from feature-map design, feature-map repetitions, circuit width, and their interactions, while variational-layer entanglement topology and gate type remain secondary.



(a) Blobs (b) Circles (c) MNIST–HQNN

Figure 24: Additional factor-wise attribution panels for the generalization gap. Together with Fig. 8, these results show that feature-map design, repetitions, circuit width, and their interactions dominate the structural gap, while variational-layer entanglement topology and gate type contribute less.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state the paper's scope as a controlled structural privacy audit of QML models, with claims tied to encoder choice, feature-map repetitions, circuit width, and MIA validation. The limitations section explicitly qualifies the scope by noting that the hierarchy may not generalize to all architectures, datasets, or hardware backends.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The paper includes a dedicated Limitations section. It discusses the controlled noiseless TorchQuantum simulation setting, possible modulation by hardware noise, the use of the train–test gap as a privacy-risk proxy, and future work on broader attacks and direct geometry measurements.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation, while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [N/A]

   Justification: The paper does not present formal theoretical results, theorems, or proofs. The mathematical expressions are used to define the encoder-induced representation, Hilbert–Schmidt kernel, and structural privacy pathway.

   Guidelines:

   - The answer [N/A] means that the paper does not include theoretical results.
   - All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
   - All assumptions should be clearly stated or referenced in the statement of any theorems.
   - The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
   - Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
   - Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: The paper discloses the experimental design, model families, structural sweep variables, preprocessing choices, feature-map templates, simulation backend, training protocol, and MIA protocol in the main text and Appendix A. The released code provides scripts and configuration files for reproducing the main experiments on synthetic datasets and MNIST.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
   - If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
   - Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
   - While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
     (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
     (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
     (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: We release our code, see Section 3: Hypothesis and Controlled Experimental Design.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The paper specifies datasets, model families, feature maps, structural sweep variables, training protocol, simulation backend, and MIA protocol. Additional details on tiled encoding, padding, MNIST preprocessing, and architecture-specific pipelines are provided in Appendix A, and the released code includes the corresponding configurations.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

Justification: The paper reports controlled structural sweeps, correlation analyses, attribution results, and direct MIA validation across multiple datasets and QML architectures, but it does not report error bars or confidence intervals for all main figures. Full multi-seed repetition of every configuration would be computationally expensive because the study varies encoder family, repetitions, width, depth, entanglement topology, gate family, model architecture, and MIA evaluation; instead, the main claims are supported through repeated evidence across controlled settings, additional appendix panels, complete structural-regime attack metrics, and correlation/attribution analyses.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [No]

   Justification: The paper states that experiments use controlled TorchQuantum simulation on DGX/A100 GPU servers, but it does not yet provide detailed per-run runtime, total GPU-hours, or memory usage for each experiment.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: The work conforms to the NeurIPS Code of Ethics. It uses public benchmark datasets and synthetic data, does not involve human subjects or private user data, and studies privacy risk in order to support safer QML design.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The paper discusses privacy risks in QML and frames QuRiFT as an auditing tool for identifying architecture-induced leakage. The positive impact is improved privacy-aware QML design; the potential negative impact is that MIA analysis could inform stronger attacks, which the paper mitigates by emphasizing auditing, design guidance, and limitations.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release high-risk pretrained models, scraped datasets, generative models, or sensitive data assets. The work uses public/synthetic benchmarks and focuses on privacy auditing rather than deployment of a high-risk model.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.

- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: Existing assets are credited in the paper and code documentation. We use publicly available MNIST data, synthetic datasets generated programmatically, and open-source software libraries including TorchQuantum, Qiskit, PyTorch, and standard Python scientific-computing packages, following their respective licenses and terms of use.

    Guidelines:

    - The answer [N/A] means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The paper releases the QuRiFT codebase as a new research artifact. The code documentation describes the implementation structure, setup instructions, configuration files, dataset preparation, structural sweeps, training/evaluation scripts, and plotting/privacy-analysis routines.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

Justification: The paper does not involve crowdsourcing, user studies, or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The paper does not involve human subjects research, crowdsourcing, or collection of personal data, so IRB approval is not applicable.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs were used only for writing assistance, including grammar correction, sentence readability, and formatting.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.